

Laboratory work No. 4
«Numerical Integration &
Differentiation 24.44»

Submitted by
Farrakhetdinova Azaliya
26/12/2025

Problem statement

A jet fighter's position on an aircraft carrier's runway was timed during landing:

t, s	0	0.52	1.04	1.75	2.37	3.25	3.83
x, m	153	185	208	249	261	271	273

where x is the distance from the end of the carrier. Estimate (a) velocity (dx/dt) and (b) acceleration (dv/dt) using numerical differentiation.

Mathematical model

Mathematical model of the problem consists of the first differentiation of coordinate (velocity) and velocity (acceleration):

$$v = \frac{dx}{dt} , \quad a = \frac{dv}{dt}$$

Numerical method

The finite difference method is one of the simplest methods for numerical differentiation. It uses the values of the function at certain points to estimate the derivative. The simple forward difference formula is defined as:

$$f'(x) \approx (f(x + h) - f(x)) / h$$

The central difference method, considered the most accurate of the simple finite difference formulas, uses points on either side of the desired value:

$$f'(x) \approx (f(x + h_1) - f(x - h_2)) / (h_1 + h_2)$$

Result of numerical solution

The main window of interface is shown on Fig.1. There are the input fields, the solution, and the graph output for visual representation of solution. There are two legends with velocity and acceleration to analyze results.

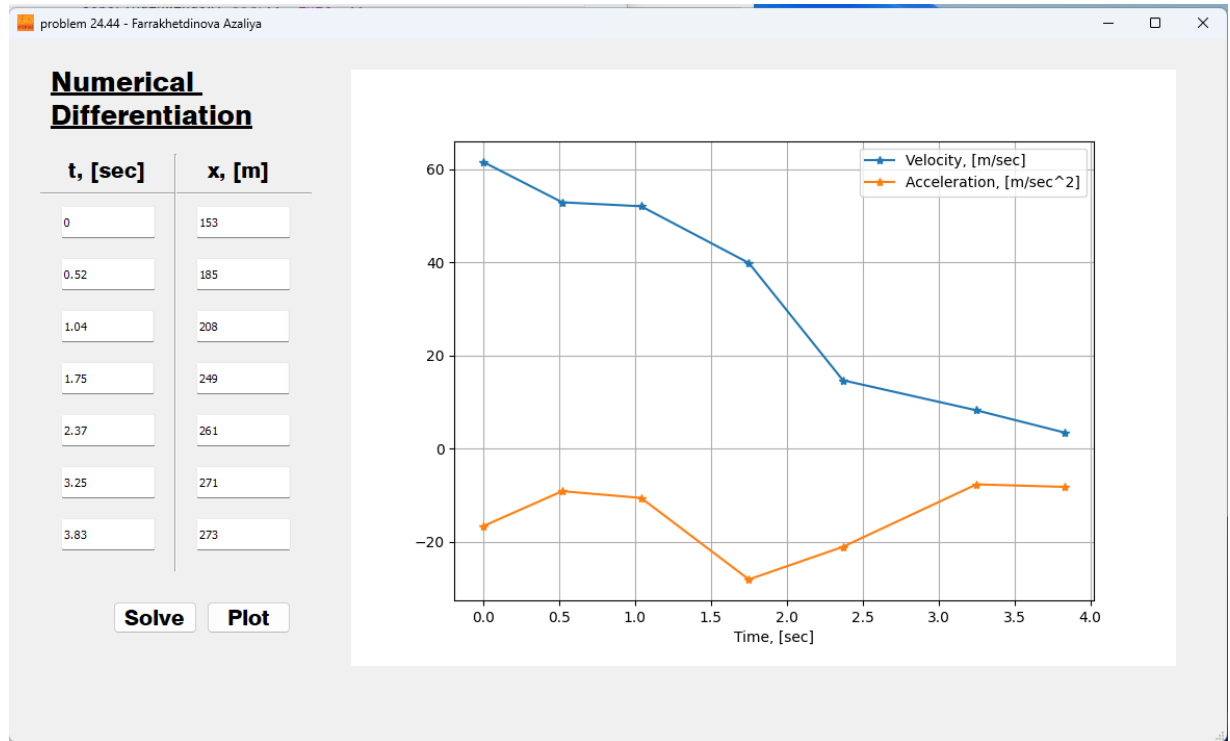


Fig.1 – Main window

Conclusion

The graph shows the dependence of acceleration on velocity. We can see that velocity decreases over time, therefore, acceleration is always negative. When velocity changes rapidly, then absolute value of acceleration is higher, and vice versa.

Appendix.

A. Main Window Code

```
import numpy as np
from PyQt5 import QtWidgets, uic

class MainWindow(QtWidgets.QMainWindow):

    def __init__(self):

        super(MainWindow, self).__init__()

        uic.loadUi('interface.ui', self)

        self.solve_button.clicked.connect(self.show_solution)
        self.plot_button.clicked.connect(self.plot_data)

    def plot_data(self):

        T = self.t
        V = self.v
        A = self.a

        self.widget_graph.canvas.axes.clear()
        self.widget_graph.canvas.axes.plot(T, V, "--", label='Velocity,
[m/sec]')
        self.widget_graph.canvas.axes.plot(T, A, "--", label='Acceleration,
[m/sec^2]')
        self.widget_graph.canvas.axes.legend(loc='upper right')
        self.widget_graph.canvas.axes.set_xlabel('Time, [sec]')
        self.widget_graph.canvas.axes.grid(True)

        self.widget_graph.canvas.draw()

    def show_solution(self):

        n = 7
        self.t: np.ndarray = np.zeros(shape=n, dtype=float)

        self.t[0] = float(self.t1.text())
        self.t[1] = float(self.t2.text())
        self.t[2] = float(self.t3.text())
        self.t[3] = float(self.t4.text())
        self.t[4] = float(self.t5.text())
        self.t[5] = float(self.t6.text())
        self.t[6] = float(self.t7.text())

        self.x: np.ndarray = np.zeros(shape=n, dtype=float)

        self.x[0] = float(self.x1.text())
        self.x[1] = float(self.x2.text())
        self.x[2] = float(self.x3.text())
        self.x[3] = float(self.x4.text())
        self.x[4] = float(self.x5.text())
        self.x[5] = float(self.x6.text())
        self.x[6] = float(self.x7.text())

        self.v = np.zeros(n)

        self.v[0] = (self.x[1] - self.x[0]) / (self.t[1] - self.t[0])
        for i in range(1, n - 1):
```

```

        self.v[i] = (self.x[i + 1] - self.x[i - 1]) / (self.t[i + 1] -
self.t[i - 1])
        self.v[n - 1] = (self.x[n - 1] - self.x[n - 2]) / (self.t[n - 1] -
self.t[n - 2])

        self.a = np.zeros(n)

        self.a[0] = (self.v[1] - self.v[0]) / (self.t[1] - self.t[0])
        for i in range(1, n - 1):
            self.a[i] = (self.v[i + 1] - self.v[i - 1]) / (self.t[i + 1] -
self.t[i - 1])
        self.a[n - 1] = (self.v[n - 1] - self.v[n - 2]) / (self.t[n - 1] -
self.t[n - 2])

if __name__ == '__main__':
    app = QtWidgets.QApplication([])
    main = MainWindow()
    main.show()
    app.exec_()

```

B. Plotting Class (using Matplotlib)

```
import matplotlib
matplotlib.use('Qt5Agg')

from PyQt5 import QtWidgets

from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure

class MplCanvas(FigureCanvasQTAgg):

    def __init__(self):
        fig = Figure()
        self.axes = fig.add_subplot(111)
        super().__init__(fig)

class MplWidget(QtWidgets.QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.canvas = MplCanvas()
        self.layout = QtWidgets.QVBoxLayout()
        self.layout.addWidget(self.canvas)
        self.setLayout(self.layout)
```